A Reproducible Data Workflow on the Titanic Dataset

John Laine

Udacity

Capstone Project - AI Programming Foundations

A Reproducible Data Workflow on the Titanic Dataset

The main body of your paper goes here, starting on page 3. (Or starting on page 2, if you're not required to include an abstract.)

Overview

This report describes a reusable; end-to-end data workflow built in a Jupyter notebook against the Kaggle 'Titanic - Machine Learning from Disaster' dataset. The notebook loads the data, applies two reusable cleaning functions, runs a structured exploratory analysis function, produces three labeled visualizations, and ends with a written interpretation. The work is intentionally framed as a foundation for later machine learning, deep learning, and agentic projects, so it follows reproducibility practices that allow another reviewer to rerun the analysis end-to-end (Danchev, 2002)

Dataset Description

The dataset is the Kaggle Titanic 'train.csv' file, which contains 891 passenger records and 12 columns. Each row represents a single passenger and includes demographic fields ('Sex', 'Age'), ticketing fields ('Pclass', 'Fare', 'Cabin', 'Ticket', 'Embarked'), family-structure fields ('SibSp' for siblings or spouse aboard, 'Parch' for parents or children aboard), and the binary outcome 'Survived'. The data has well-known quality issues: 'Age' is missing for roughly 20% of rows, 'Cabin' is missing for roughly 77%, and 'Embarked' is missing for two rows. The sample is the Kaggle training split rather than a complete passenger manifest – only 891 of the

roughly 2,224 people on board – which is an important context for any survival rate I report later.

Workflow Description

Ingestion

Data is loaded with ‘pandas.read_csv’ from a relative path (‘./data/titanic/train.csv’) so the notebook runs without reconfiguration on any machine that has the repository cloned. The first cells confirm the load with ‘df.head()’.

Cleaning

Cleaning is implemented as two reusable functions, ‘handle_missing_values(df)’ and ‘standardize_features(df)’. Both return a new DataFrame instead of mutating in place, which makes the original ‘df’ available for before-and-after comparison. The first function imputes ‘Age’ using the within-group median by ‘Pclass’ and ‘Sex’, fills the two missing ‘Embarked’ values with the model port, and replaces the sparse ‘Cabin’ column with a binary ‘HasCabin’ indicator. The second function casts ‘Sex’, ‘Embarked’, and ‘Pclass’ to categorical dtypes, expands the port codes (‘C’, ‘Q’, ‘S’) to their full names, and engineers two new features: ‘FamilySize = (SibSp + Parch + 1)’ and ‘IsAlone = (FamilySize ==1)’. Organizing each variable as a single column with a consistent meaning is a deliberate tidy-data choice that makes the subsequent grouping and plotting code straightforward (Wickham, 2014).

Exploratory Analysis

A single function, `explore_dataset(df, target, group_cols)`, produces the structured EDA: shape, dtypes, numeric `describe()`, target-mean breakdowns over each grouping column, and a Pearson correlation matrix that is returned for reuse. Packaging EDA as a function rather

than a sequence of ad-hoc cells means the same analysis can be rerun on any subset of the data (for example, only first-class passengers) without duplicating code.

Visualizations

Three labeled plots are produced, each with a title, labeled axes, and an interpretation cell. Figure 1 is a grouped bar chart of survival rate by `Pclass` and `Sex`. Figure 2 is an overlaid histogram with KDE comparing the `Age` distributions of survivors and non-survivors. Figure 3 is a heatmap of the correlation matrix returned by `explore_dataset`.

Summary

The notebook closes with a written summary that lists what I learned, the most informative patterns (linking back to each figure), and the assumptions and limitations behind the cleaning and the dataset itself.

Key Decisions and Assumptions

Cleaning choices

The most consequential cleaning decision was how to handle the three missing-value patterns. Imputing `Age` within `Pclass` and `Sex` groups was chosen because median age varies meaningfully across those groups; a single global median would have erased that structure. Filling `Embarked` with the modal port (Southampton) was appropriate because only two rows are affected, so the impact of any imputation choice is negligible. The `Cabin` decision is the most defensible-but-contestable choice: rather than drop the column (information loss) or impute 77% of values (fabrication), I converted it to a `HasCabin` indicator, which preserves the signal carried by the *presence* of a cabin record. These choices assume the missingness mechanism is at least conditionally random given class and sex, which is a known assumption underlying

group-mean imputation and one that practitioners are routinely warned to make explicit (Danchev, 2022).

EDA focus

The EDA function deliberately surfaces three things: distributional summaries (`describe`), categorical breakdowns of the survival rate (`groupby(col)['Survived'].mean()`), and a numeric correlation matrix. These match the three questions a reviewer is most likely to ask of this dataset: how is each variable distributed, which groups survived more often, and which features move together?

What each plot was designed to show

Figure 1 was chosen to show the joint effect of class and sex on survival, since these are the two strongest single predictors and they interact. Figure 2 was chosen to test whether age — and specifically being a child — added information on top of class and sex. Figure 3 was chosen to give a one-glance overview of which numeric features correlate with `Survived` and to flag redundancies created by feature engineering (for example, `FamilySize` is by construction nearly collinear with `SibSp` and `Parch`).

Results and Interpretation

The cleaned dataset has zero missing values across the columns retained for analysis, and the engineered features behave as expected: `FamilySize` ranges from 1 to 11, and `IsAlone` is true for about 60% of passengers.

Figure 1 shows the joint effect of class and sex on survival. First-class women survived at a rate near 100%, while third-class men survived at a rate of roughly 15%. Sex is the stronger of

the two factors — women out survived men in every class — but class shifts the survival rate by a large margin within each sex. This is consistent with the historical record of women-and-children-first lifeboat loading combined with the physical layout of the ship, which made it harder for lower-deck passengers to reach the boats.

Figure 2 shows a smaller but real age effect. The two age distributions overlap heavily through the working-age range, but there is a visible bump in the survivor density for children under roughly ten years old. Age alone is therefore a weaker predictor than sex or class, but it adds signal at the extremes.

Figure 3 quantifies the patterns numerically. `Fare` (Pearson r ≈ 0.26) and `HasCabin` (r ≈ 0.32) are the strongest numeric correlates of `Survived`, both of which are class proxies — wealthier passengers paid more and were more likely to have a cabin record. `FamilySize` correlates very strongly with `SibSp` (r ≈ 0.89) and `Parch` (r ≈ 0.78) by construction, which would need to be addressed by a feature selection step before modeling.

The most surprising finding was the size of the `HasCabin` effect even after accounting for class: passengers with *any* cabin record survived at noticeably higher rates than those without, which suggests the missingness in `Cabin` is itself informative and not neutral.

Responsible Practice (Bias and Data Quality)

Several decisions in this workflow could introduce or amplify bias if applied uncritically. Group-mean imputation of `Age` shrinks within-group variance and reinforces the demographic

structure already present in the data; a downstream model trained on this version of `Age` could exaggerate any class- and sex-linked age stereotype. Converting `Cabin` to a binary `HasCabin` flag treats missingness as informative, which is empirically justified here but also embeds historical inequities in record-keeping (lower-class passengers may simply have had less complete documentation) directly into a model feature. Filling `Embarked` with the modal port silently erases the experience of passengers from less common embarkation points. These are exactly the kinds of decisions that prior work on algorithmic fairness identifies as common sources of disparate impact: choices made for technical convenience during cleaning can encode and amplify pre-existing inequalities in the source data (Barocas & Selbst, 2016).

A second responsibility issue is the dataset itself. The 891-row Kaggle training split is a sample of the roughly 2,224 people on board, and it is the sample chosen for a competition rather than a complete manifest. Any survival rate reported here is an estimate, not ground truth, and should be presented that way.

Reproducibility

Reproducibility was treated as a first-class concern from the outset, in line with widely accepted guidance on reproducible Python workflows (Danchev, 2022). Three concrete practices support this in the repository:

1. A `requirements.txt` file pins the exact versions of `numpy`, `pandas`, `matplotlib`, `seaborn`, and `jupyter`. A reviewer can recreate the environment with `pip install -r requirements.txt`.

2. The notebook reads the dataset from a relative path stored inside the repository (`./data/titanic/train.csv`), so it runs without any environment-specific configuration.
3. The Git workflow uses a `main` branch plus at least one development branch, with regular commits as each task was completed, making the history of decisions auditable.

Packaging cleaning and EDA as functions (rather than ad-hoc scripts) also supports reproducibility: the same function calls applied to a fresh download of the data will produce the same cleaned DataFrame and the same plots.

Sources and Citations

Barocas, S., & Selbst, A. D. (2016). Big data's disparate impact. *California Law Review*, *104*(3), 671–732. https://doi.org/10.15779/Z38BG31

Danchev, V. (2022). Reproducible data science with Python: An open learning resource. *Journal of Open Source Education*, *5*(56), 156. https://doi.org/10.21105/jose.00156

Wickham, H. (2014). Tidy data. *Journal of Statistical Software*, *59*(10), 1–23. https://doi.org/10.18637/jss.v059.i10